TECHNO-FUNCTIONAL REQUIREMENTS DOCUMENT (TFRD)

Project Title:	Scalable, Self-Hosted Search & Scraping Pipeline for Agentic AI
Version:	1.0.0
Date:	May 21, 2026
Status:	Approved / Architecture Design Phase

1. Executive Summary & Objective

1.1 Objective

The objective of this document is to define the technical and functional requirements for deploying a fully self-hosted, scalable search and web-scraping infrastructure. This system serves as a cost-free, open-source replacement for paid third-party search APIs (such as SerpAPI) and managed extraction services (such as Firecrawl Cloud).

1.2 Context & Scale

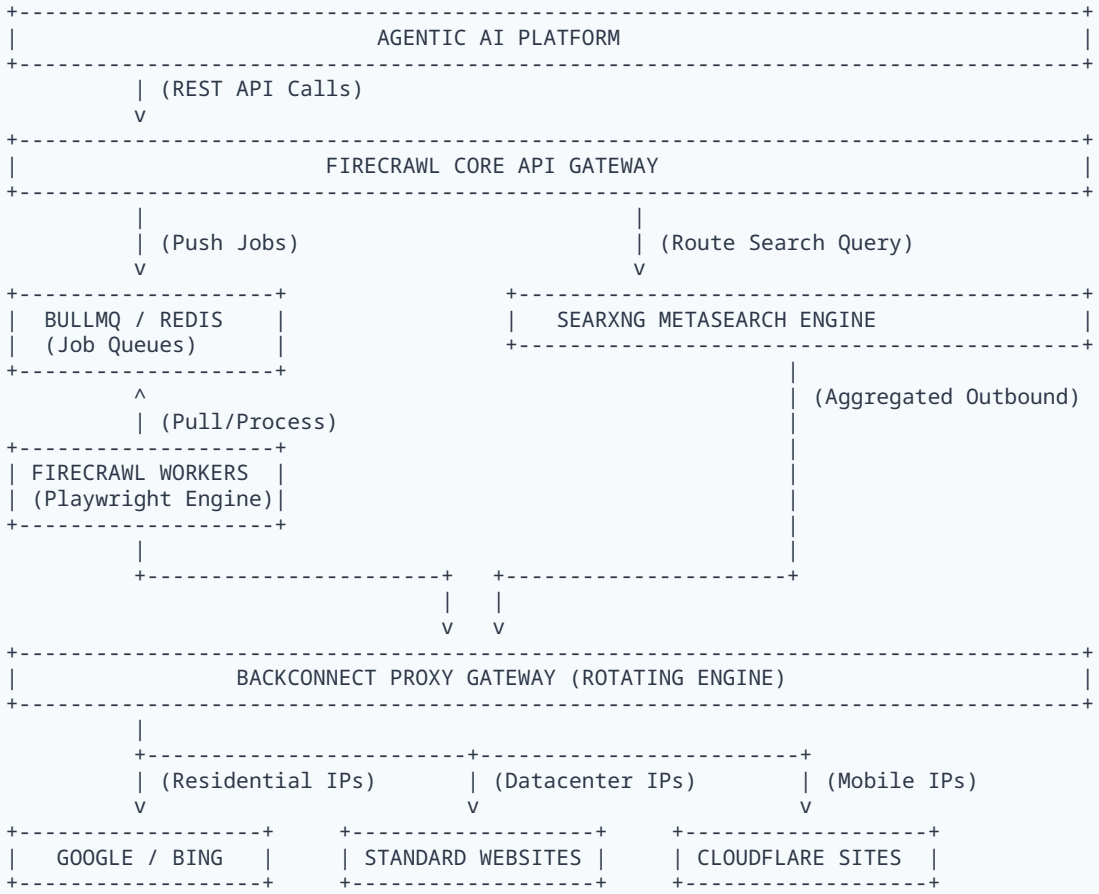
The infrastructure will power an Agentic AI Platform supporting **10–20 concurrent deep research agents**. Each agent executes continuous, long-running research tasks containing nested iterative loops. The target system load is characterized by:

- **Concurrent Active Agents:** 15 agents average (20 peak).
- **Search Volume:** ~300 search queries per hour platform-wide.
- **Scraping Volume:** ~900 target web page extractions per hour platform-wide.
- **Peak Burst Concurrency:** 40 to 60 simultaneous headless browser sessions.

2. System Architecture & Component Interaction

The infrastructure follows a decoupled microservices architecture orchestrated via containerization. It comprises an orchestration API layer, an asynchronous distributed job queue, an isolated browser engine pool, a metasearch aggregation engine, and an upstream outbound backconnect proxy gateway.

FIGURE 2.1: DATA PIPELINE COMPONENT INTERACTION



2.1 Core Architectural Flow

1. **Initiation:** An AI Agent sends an HTTP POST request to `/search` or `/scrape` on the **Firecrawl Core API Gateway**.
2. **Search Processing:** If a `/search` request is issued, Firecrawl delegates the query parsing to the **SearXNG** engine. SearXNG translates the query, strips user-identifiable parameters, and passes it through the **Backconnect Proxy Gateway** to fetch results concurrently from Google and Bing.
3. **Queueing & Scraping:** When URLs are returned (or if a direct `/scrape` request is received), jobs are sent to **BullMQ (Redis)**. **Firecrawl Workers** consume these tasks, initializing isolated **Playwright (Chromium)** instances.
4. **Proxying and Extraction:** The Playwright instance routes its network traffic through the Backconnect Proxy Gateway, applying specific rotation rules. It renders the target DOM, extracts structural contents, converts it into clean Markdown, and returns the payload to the calling agent.

3. Functional Requirements (FR)

- **FR-1.1 Search Endpoint (`/search`):** The system must accept a natural language text query and an integer limit parameter. It must return a clean JSON array containing titles, URLs, and text snippets aggregated from Google and Bing.
- **FR-1.2 Scrape Endpoint (`/scrape`):** The system must accept a target URL, render client-side JavaScript, and return structural data converted directly into LLM-readable Markdown format.
- **FR-1.3 Crawl Endpoint (`/crawl`):** The system must support asynchronous deep crawling of a base domain to a maximum specified depth, populating a queue and returning text map configurations.
- **FR-2.1 HTML-to-Markdown Parser:** The system must strip non-semantic HTML nodes (`<nav>`, `<footer>`, `<script>`, `<style>`, `<iframe>`, ad tracking banners) prior to markdown conversion.
- **FR-2.2 Metadata Extraction:** The extraction engine must output a consistent JSON block containing top-level page metadata: `title`, `description`, `author`, `publishDate`, and `canonicalUrl`.
- **FR-3.1 Engines Supported:** The backend metasearch component must simultaneously query Google (Web, News) and Bing.
- **FR-3.2 De-duplication and Ranking:** Results from overlapping search providers must be dynamically de-duplicated based on normalized URL matching and re-ranked into a single output list.
- **FR-4.1 Session Persistence:** The system must accept a `sessionId` header parameter to assign a "Sticky Session," locking the outbound IP address for up to 10 minutes to allow continuous navigation.

4. Technical & Non-Functional Requirements (NFR)

4.1 Performance, Concurrency, & Scale

- **NFR-1.1 Target Concurrency:** The infrastructure must process a baseline of 30 concurrent web-scraping sessions and scale to a maximum burst capacity of 60 concurrent browser tabs.
- **NFR-1.2 API Latency Target:** SearXNG-only raw searches must resolve within ≤ 2.0 seconds. Full JavaScript page extractions must resolve within ≤ 6.0 seconds.

4.2 Queue Management & Resiliency

- **NFR-2.1 Job Isolation via Redis:** All tasks must be wrapped as atomic jobs inside a BullMQ state machine running on Redis.
- **NFR-2.2 Concurrency Throttling:** Environment limits (`MAX_CONCURRENCY=30`) must force overflowing queries to wait inside the queue rather than crashing the operating system.
- **NFR-2.3 Auto-Retry Policies:** Failed requests caused by timeouts or $5xx$ codes must automatically retry up to 3 times with exponential backoff ($2^n \times 1000ms$).

4.3 Network Optimization & Proxy Slicing

- **NFR-3.1 Data Minimization Policy:** To minimize proxy costs, the Playwright engine **must** intercept and abort requests for non-textual assets (images, fonts, videos, ad networks).

- **NFR-3.2 Header Emulation:** Outbound browser calls must randomize HTTP headers (`User-Agent`, `Accept-Language`) to mimic active commercial browsers.

4.4 Tiered Routing Strategy

Target Site Class	Routing Mechanism	Proxy Tier Used
Search Engines (Google/Bing)	SearXNG Outbound Router	Rotating Residential Proxy
WAF / Cloudflare Protected Domains	Firecrawl Playwright Pool	Rotating Residential / Mobile Proxy
Standard Blogs, Docs, Public Wikis	Firecrawl Playwright Pool	Datacenter Proxy Pool (Unlimited)

5. Infrastructure & Deployment Specification

5.1 Technology Stack Matrix

- **Core Scraper Engine:** Firecrawl Open-Source (Node.js, TypeScript, Playwright).
- **Metasearch Engine:** SearXNG (Python-based dockerized metasearch engine).
- **Job Broker & Storage:** Redis (v7.2+) coupled with BullMQ.
- **Containerization:** Docker CE / Docker Compose Engine v2.x.

5.2 Recommended Infrastructure Blueprint

Host Node A: Orchestration, API, and Cache (1 Node)
Specs: 4 vCPUs, 16 GB RAM (e.g., AWS `c6i.xlarge`). Services: `firecrawl-api-gateway`, `searxng-engine`, `redis-cache-queue`.

Host Node B: Distributed Browser Workers (2 Nodes - Shared Load)
Specs Per Node: 8 vCPUs, 32 GB RAM (e.g., AWS `m6i.2xlarge`). Services: `firecrawl-worker` instances capped at 20 concurrent headless browser workers maximum per node.

5.3 Configuration Environment Blueprint

SearXNG Configuration (`settings.yml` snippet):

```
outgoing:
  request_timeout: 4.0
  max_request_timeout: 8.0
  proxies:
    http: http://user:pass@gate.residential-proxy.com:8000
    https: http://user:pass@gate.residential-proxy.com:8000
```

Firecrawl Worker Configuration (`.env` snippet):

```
TOTAL_WORKERS_PER_NODE=20
MAX_CONCURRENCY=20
PROXY_SERVER=http://gate.datacenter-proxy.com:1000
RESIDENTIAL_PROXY_SERVER=http://gate.residential-proxy.com:8000
BLOCK_IMAGES=true
BLOCK_FONTS=true
PAGE_TIMEOUT_MS=30000
```

6. Risk Mitigation & Error Handling Protocols

- **6.1 DOM Drift Anomaly Protocol:** If a successful HTTP 200 response yields zero array nodes while querying high-volume terms, trigger an automated alert and fail over routing completely to the alternate search engine.
- **6.2 CAPTCHA & Anti-Bot Interception:** Upon receiving a 403 or 429 status, the system must discard the current proxy gateway session ID and automatically spin up a fresh session thread with a localized user-agent shift.
- **6.3 Memory Leaks & OOM Prevention:** Enforce an automated process lifecycle loop inside Firecrawl where each worker process automatically terminates and respawes a clean browser daemon after executing a maximum of 50 consecutive tasks (`WORKER_LIFECYCLE_MAX_JOBS=50`).